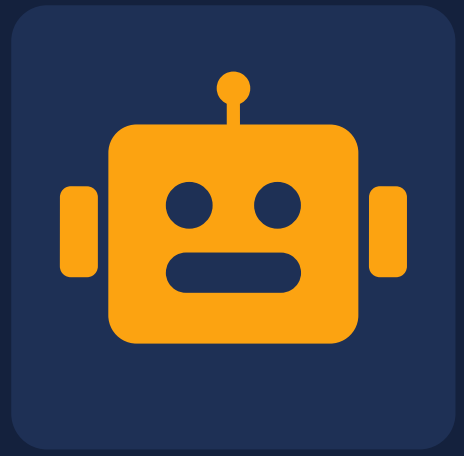
AULA 4 · ETAPA 3 · TURMA LIVE · 1H30

Clínica do TP1 — e as interfaces do seu projeto

Última aula antes da entrega (sexta 13/11): primeiro aterrissar o TP1, depois o vocabulário próprio do TP2



Prof. Dácio Moreira de Souza · segunda 09/11 · Zoom · turma GRLEDCR3C2-N2-L1

CONFERÊNCIA (8 MIN)

Onde o seu TP1 está

1	**Parte A na dev: pacote seu, taxas interpretadas, PROJETO.md com o G1.1**	*G1.0 · G1.1 · G1.2*
2	**Parte B na dev: duas classes, YAML, /vision/status com domínio, oclusão**	*G1.3 · G1.4*
3	**PROJETO.md com os gates marcados — [x], [ ] ou [!]**	*bloco PB:GATES-TP1*
4	**Evidências em docs/evidencias/aula03/ (e cópias em tp1/)**	*5 arquivos*
5	**Relatório começado: docs/relatorios/relatorio-tp1.md**	*G1.5 · qua 11/11*

Ponto de controle 1 — cole as três saídas no chat:

```
grep -c "PB:PROJETO\|PB:TRILHA" PROJETO.md  ·  grep -o "\[x\] G1\.." PROJETO.md  ·  ls docs/evidencias/{aula03,tp1}
```

Sem ambiente ou sem repositório a 4 dias da entrega? Escreva o sintoma: hoje a resposta é a escada de recuperação — entregue o que funciona e declare o que não funciona.

A AULA EM UMA FRASE

Duas aulas dentro de uma

A primeira metade é clínica: a entrega é sexta. A segunda é o começo do TP2: o seu projeto para de falar a língua dos outros.

Mensagem pronta — Int32, Trigger, Image — foi a escolha certa até aqui: começar inventando tipo é uma forma cara de não entregar nada. Hoje vocês veem o limite exato dela, e a resposta.

Se tiver que escolher onde prestar atenção, escolha a primeira metade. O conteúdo novo tem o guia de estudo; a entrega de sexta não tem substituto.

ROTEIRO

Como os 90 minutos se dividem

Minuto	Bloco	O que acontece
0–8	Conferência	ponto de controle 1: onde o TP1 de cada um está
8–40	Clínica do TP1	censo de gates, os 3 bloqueios e a cura de cada um, relatório em 5 perguntas, ensaio da entrega, 2 telas compartilhadas
40–65	Interfaces próprias	três o quê? · o pacote separado · anatomia de .msg e .srv · serviço × ação — com 1 demo direta
65–80	Exemplo de ponta a ponta	no pb-live-dacioms: dois pacotes, build na ordem, interface show, launch, echo, service call, a confiança caindo
80–90	Até sexta — e depois	relatório qua 11/11, checklist sex 13/11; guia e tarefa da Aula 4 depois (dom 22/11); Aula 5

A clínica não cede tempo. Se a aula atrasar, quem encurta é a Parte 2 — o guia de estudo repete tudo com as saídas esperadas.

PARTE 1 · 8–40 MIN

Clínica do TP1

entrega: sexta 13/11

- censo de gates: onde cada um se prova
- os três bloqueios — e a cura de cada um
- o relatório em cinco perguntas
- ensaio da entrega, escada de recuperação, duas telas

CENSO DE GATES

Onde cada gate se prova — e o que fica no repositório

Gate	Como se prova	Evidência · prazo
G1.0	bash scripts/check-ambiente.sh → 'Tudo pronto!'	docs/evidencias/tp1/check-ambiente.txt
G1.1	grep -c "PB:PROJETO|PB:TRILHA" PROJETO.md → 2; DUAS classes	PROJETO.md
G1.2	ros2 topic hz nas duas pontas, interpretado	docs/evidencias/tp1/topic-hz.txt
G1.3	echo /vision/contagem com as duas classes da SUA cena	docs/evidencias/tp1/segmentacao.png
G1.4	service call /vision/status com domínio + rqt_graph	rqt_graph.png + status.txt
G1.5	relatório com as 5 seções preenchidas	docs/relatorios/relatorio-tp1.md · qua 11/11
G1.6	git tag --list → tp1; Moodle com PDF, ZIP e links	tag tp1 + Moodle · sex 13/11

Enquete no chat: qual o MENOR gate que ainda está aberto no seu projeto (G1.0 … G1.6, ou 'nenhum')? É o censo que ordena os próximos 25 minutos.

OS TRÊS BLOQUEIOS

O que mais trava a entrega — e a cura de cada um

1	**Projeto com UMA classe só (G1.1) — o mais grave e o mais rápido: 15 min de conversa**	*domínio → duas classes*
2	**/vision/status genérico (G1.4) — dez linhas; gabarito no guia da Aula 3, seção 3**	*string com domínio*
3	**Evidência que ficou no terminal — rode de novo e redirecione para arquivo**	*o terminal fecha*
4	**Oclusão não registrada — echo da contagem, um print, três linhas**	*observação, não defeito*

A cura do bloqueio 3, em uma linha (pipeline no ar):

```
ros2 service call /vision/status std_srvs/srv/Trigger "{}" > docs/evidencias/tp1/status.txt
```

…e depois git add -A && git commit -m "G1.4: evidencias" && git push origin dev. O terminal fecha; o repositório fica.

O RELATÓRIO

Documento de decisões, não diário

Diário — o que trava

não faça

- narrar o mês em ordem cronológica
- colar logs sem interpretar
- esconder o que não funcionou
- esquecer de declarar a IA

Decisões — cinco perguntas

faça

- domínio e derivação: que problema, de quem, por que este
- decisões técnicas: o que escolheu E o que descartou
- oclusão: o que aconteceu e o que implica no domínio
- limitações: o que o sistema NÃO faz, dito por você
- IA: o que foi gerado, revisado, e o que você sabe explicar

A seção de limitações é a que mais gente pula e a que mais revela entendimento. Modelo: docs/relatorios/relatorio-template.md no seu repositório. Prazo recomendado: quarta 11/11.

ENSAIO DA ENTREGA

Cinco passos — hoje, não na sexta

1	**main atualizada: dev mesclada, push**	*git checkout main && git merge dev*
2	**branch entrega-tp1 criada e empurrada**	*git checkout -b entrega-tp1*
3	**tag tp1 anotada e empurrada — congela o código avaliado**	*git tag -a tp1 · git push origin tp1*
4	**ZIP do código SEM build/, install/, log/, .git/ + PDF do relatório**	*Moodle*
5	**todos os links em janela anônima: repositório, vídeo (≤5 min, não-listado), PDF**	*se não abre, não existe*

Passo a passo com os comandos: tutoriais/checklist-entrega-tp1 — leia de cima para baixo num dia em que ainda dá tempo de consertar.

```
git tag -a tp1 -m "Entrega do TP1" && git push origin tp1    # depois de main atualizada e entrega-tp1 criada
```

Commit depois da tag não é considerado — e é isso que protege você: o que veio depois não conta contra.

A ESCADA

Não vai dar tempo de tudo? Entregue o que funciona e declare o que não funciona

Degrau 1: reduza o escopo do gate — cena menor, fonte sintética, uma classe a menos — sem abandoná-lo. Degrau 2: marque [!] no PROJETO.md e escreva a limitação no relatório. Degrau 3: entregue no prazo com o que há.

Limitação escrita vale mais do que falha descoberta. Um [!] declarado custa pouco; um [x] sem evidência é fácil de verificar — e um repositório vazio na sexta não tem degrau nenhum.

Ponto de controle 2 (chat): 'até sexta me falta: G1.x, G1.y' — ou 'nada'. É a sua lista, não a minha. Fila para as duas telas compartilhadas: quem está travado em código, agora.

Mentoria por tela: 5 minutos cada; o que passar disso vira mensagem no Infnet.Online com os quatro itens.

PARTE 2 · 40–65 MIN

Quando a mensagem pronta acaba

em linhas gerais — o detalhe está no guia (depois de sexta)

- dois o quê? a informação jogada fora
- o pacote separado — e por quê
- anatomia de um .msg e de um .srv (demo direta)
- onde o serviço deixa de servir

DOIS O QUÊ?

A informação existe — e é jogada fora na publicação

ros2 topic echo /vision/contagem --once → data: 2. Dois o quê? De que classe? Com que confiança? Onde na imagem? De que instante?

O segmentador conhece a área, a posição e o contorno de cada objeto — e publica um Int32. Não é limitação do ROS; é escolha de tipo. O tipo pronto serviu enquanto a pergunta era 'quantos?'; deixa de servir quando a pergunta vira 'quantos, de quê, com que certeza, e onde?'.

Regra prática: três ou mais campos que andam sempre juntos → interface própria. Um número solto → Int32 continua certo. JSON dentro de String → nunca: contrato vira convenção que só existe na sua cabeça.

O PACOTE SEPARADO

Por que interfaces vivem num pacote ament_cmake próprio

<projeto>_interfaces · ament_cmake

só .msg, .srv e .action — nenhum nó

- rosidl gera código C++ e Python a partir dos .msg
- 3 linhas do package.xml fazem isso acontecer
- compila PRIMEIRO, source DEPOIS
- é exatamente o que sensor_msgs é

<projeto>_percepcao · ament_python

os seus nós, como desde a Aula 2

- <depend><projeto>_interfaces</depend> no package.xml
- from <projeto>_interfaces.msg import …
- não tem a maquinaria do rosidl
- compila depois, com --symlink-install

Gerar interface é gerar código. Um pacote Python não tem essa maquinaria — e o erro de tentar é um CMake reclamando onde você nem sabia que havia CMake.

ANATOMIA

Deteccao · Deteccoes · StatusVisao

Um .msg é uma lista de campos — tipo, nome, comentário. Mensagens compõem; [] faz array. O .srv separa pergunta e resposta com ---.

Deteccao.msg

uma detecção individual

- string classe
- float32 confianca
- int32 x, y, area

```
classe = 'tomate_maduro'
```

Deteccoes.msg

o que a percepção viu em UM quadro

- std_msgs/Header header
- Deteccao[] deteccoes
- uint32 total

```
header.stamp · frame_id
```

StatusVisao.srv

pergunta vazia → resposta tipada

- bool ativo · string classe_alvo
- uint32 total_detectado
- float32 confianca_media · string mensagem

```
---  separa req / resp
```

Demo direta: ros2 interface show std_srvs/srv/Trigger ao lado de ros2 interface show pb_interfaces/srv/StatusVisao — o mesmo lugar do grafo, outro vocabulário.

ONDE O SERVIÇO PARA

Serviço × ação

⇄

Os dois são pergunta com resposta. A diferença é o tempo que a resposta demora — e o que acontece enquanto ela não chega.

Serviço

pergunta e resposta, agora

- quem chama fica bloqueado esperando
- 'qual o seu status?'
- resposta em fração de segundo

```
/vision/status
```

Ação

tarefa longa, com progresso

- feedback periódico enquanto roda
- cancelamento no meio
- 'vá até a bancada e me avise'

```
VarrerCena.action (Aula 5)
```

Se a resposta demora mais que uma fração de segundo, serviço é a escolha errada. Ação é a Aula 5 (16/11) — e o coração do TP2.

AO VIVO · 65–80 MIN

Dois pacotes, de ponta a ponta

>_ *A ordem importa: interfaces, source, nós. Ponto de controle 3 (quem replicou): cole a linha do monitor ou a resposta do service call.*

- `cp -r …/aula06-interfaces/pb_interfaces …/aula06_percepcao ros2_ws/src/`
- `colcon build --packages-select pb_interfaces && source install/setup.bash`
- `colcon build --packages-select aula06_percepcao --symlink-install && source install/setup.bash`
- `ros2 interface show pb_interfaces/msg/Deteccoes  # teste de fumaça`
- `ros2 launch aula06_percepcao percepcao.launch.py classe:=tomate_maduro`
- `ros2 topic echo /vision/deteccoes --once  ·  ros2 service call /vision/status pb_interfaces/srv/StatusVisao "{}"`

A PONTE PARA A ETAPA 4

A contagem cai — e a confiança cai junto

Quando os objetos se encostam, total passa a 1 e confianca despenca: o detector tem um sinal interno de que algo saiu do esperado. Antes você via só o número mudar; agora vê o detector perdendo certeza.

A confiança do exemplo é a solidez do contorno — área ÷ área do fecho convexo. Não é probabilidade, e o código diz isso em voz alta no comentário: um substituto honesto e declarado vale mais do que um número inventado com cara de probabilidade.

Um detector treinado produz esse sinal de forma direta e calibrada — é a Aula 7 (30/11). O que NÃO muda é a mensagem: classe, confiança e posição servem para os dois.

Trocar o miolo sem trocar o contrato é a razão de esta aula vir antes daquela. Interface é contrato; detector é implementação.

PARTE 3 · 80–90 MIN

Até sexta — e depois dela

entrega do TP1: sexta 13/11

- relatório até quarta 11/11 (G1.5)
- checklist de entrega na sexta: main, entrega-tp1, tag tp1, Moodle
- depois: guia de estudo da Aula 4 + tarefa até dom 22/11
- Aula 5 (16/11): ações — o pacote de interfaces ganha um .action

ATÉ SEXTA 13/11 · DEPOIS, ATÉ DOM 22/11

O que sai daqui

TP1 · sexta 13/11 · Moodle

fecha G1.5 e G1.6

- relatório: 5 perguntas, limitações ditas por você (qua 11/11)
- evidências em docs/evidencias/tp1/ (5 arquivos)
- main atualizada, branch entrega-tp1, tag tp1
- Moodle: PDF + ZIP + links; vídeo ≤ 5 min, não-listado; janela anônima

Tarefa da Aula 4 · dom 22/11 · dev

começa o TP2 — só depois de sexta

- <projeto>_interfaces (ament_cmake) compilando
- .msg com o vocabulário do SEU domínio
- /vision/status migrado para o seu .srv
- /vision/deteccoes publicando; /vision/contagem mantido

Prioridade: TP1 → guia de estudo da Aula 4 (2h–3h, duas sessões) → tarefa. Não mexa na tarefa antes de sexta — nada dela entra na entrega-tp1.

SOCORRO RÁPIDO

Os sete erros das interfaces — e um da entrega

Sintoma	Causa provável e cura
ModuleNotFoundError: No module named 'pb_interfaces'	faltou source install/setup.bash DEPOIS do build, em todos os terminais — ou a ordem de build invertida
erro de CMake num pacote onde você não escreveu CMake	msg/ dentro do pacote ament_python — interfaces vivem em pacote ament_cmake separado
compila, mas ros2 interface list não mostra nada	faltam as 3 linhas de rosidl no package.xml (o erro silencioso)
Header não resolve	std_msgs nos DOIS lugares: <depend> no package.xml e DEPENDENCIES no CMake
campo novo 'não existe'	terminal com source velho — abra um terminal novo
service call reclama do tipo	ros2 service type /vision/status e use o tipo que ele mostrar
link do repositório dá 404 para o professor	convite não aceito ou repositório fora da org — confira em janela anônima ANTES de entregar

Antes de pedir ajuda no Infnet.Online: comando exato, saída completa, lsb_release -a e ros2 interface list | grep <seuprojeto>.

PRÓXIMO ENCONTRO · SEGUNDA 16/11

Aula 5 — ações: tarefas longas, feedback e cancelamento

antes disso: TP1 entregue na sexta 13/11 · Etapa 4 começa em 16/11

- tag tp1 empurrada + PDF, ZIP e links no Moodle
- relatório com as limitações ditas por você
- interfaces: depois de sexta, até dom 22/11
- Ponto de controle 4 agora: 'TP1: sei o que me falta até sexta'